

RAGFlow 官方实践案例的工程证据审计

执行摘要

- **RAGFlow 官方最强的一条证据线不是“通用企业知识库”，而是金融投研。**原因很直接：官方不仅给了完整的 RAGFlow in Practice 实践文，还在后续 release notes 里补上了对应的 Company Research Report Deep Dive Agent 模板线索；相比之下，法律、制造、教育目前主要还是 Solution 页面，证据密度明显弱很多。¹
- **官方最常出现的数据源是“三层组合”：**内部知识库文档、外部搜索/API、结构化数据库。内部文档包括研报、产品资料、用户手册、教材、SOP、工单等；外部侧常见 Tavily Web Search、Yahoo Finance、AlphaVantage/MCP；结构化侧则是 SQL 数据库与 Execute SQL。这说明 RAGFlow 官方实践不是“纯检索”，而是 RAG + Tool/API + Structured Data 的混合编排。²
- **官方并不主张“所有场景都 Agent 化”，而是明确偏向 Workflow 与 Agent 混合。**Agentic Workflow 一文明确说，企业场景需要 deterministic 的 Workflow 与不确定输入下的 Agentic Workflow 协同；Retrieval 既能作为 workflow component，也能作为 Agent tool。这个判断在电商客服案例里被贯彻得最清楚：顶层用 Categorize 路由，叶子节点才是小型 Agent。³
- **简单、高响应、任务边界稳定的场景，官方更推荐轻量 workflow，而不是 heavy Agent。**电商客服教程给出了非常明确的反常识判断：虽然也能做成 Agent-based workflow，但 Agent 的 planning/reflection 会显著拖慢响应，因此不适合高响应、相对简单的售后客服。这个观点比很多“逢场景必上 Agent”的宣传更有工程价值。⁴
- **真正需要 Agent / multi-agent 的，是 Deep Research 这类“步骤和搜索策略本身不确定”的任务。**官方 Deep Research 模板不是多拖几个节点，而是把 Lead Agent + Web Search Specialist + Deep Content Reader + Research Synthesizer 作为角色化协作体系，并且显式支持 BFS/DFS 式计划、反思、迭代、多源搜索和长上下文综合。官方还直说：Deep Research 用拖拽 workflow 只“能跑”，但复杂度会很快失控。⁵
- **官方最值得学的一个模式，是“多知识库并行检索 + 单点生成/执行”。**SQL Assistant 与 Text2SQL 反复使用 Schema + Q->SQL + Database Description 三库并行检索，再统一喂给 Agent 生成 SQL，最后由 SQL Executor 执行；电商客服则把 Product Information 与 User Guide 分库管理，而不是把所有数据塞进一个大杂烩知识库。这是非常实用的 data structure by function 设计。⁶
- **官方在结构化输出上很重视“约束输出面”。**投研案例要求 stock code extraction 只返回股票代码或 Not Found；Deep Research 的搜索子 Agent 被要求只返回恰好 5 个 URL，以避免 attention fragmentation；SQL Assistant 明确要求 Agent 只输出一条可执行 SQL，不允许解释；安装预约 Agent 只收集三项字段。RAGFlow 的工程风格不是“让模型自由发挥”，而是尽量把输出变成受约束的中间件契约。⁷
- **官方越来越强调“RAG + Code”，而不是让 LLM 硬算。**投研案例用 Code 节点做字段映射和数值格式化，避免把财务表格生成完全交给模型；v0.25 更直接指出 LLM 擅长定性，不擅长定量，于是给出 Data Analytics 模板，通过 sandbox 执行 Python、用 matplotlib 生成图表与可下载 PNG。这个方向说明其 Context Engine 路线并不把 LLM 当万能计算器。⁸

- **引用与 traceability 在官方案例里分层明显：**金融投研案例不仅要求 citation，还要求保留机构间分歧，不能把不同观点“揉成一个结论”；制造和教育 Solution 页面也强调 grounding 到手册章节、教材章节和政策来源；但电商客服与 SQL 案例对 grounded citations 的要求明显弱得多，更关注流程正确性与结构化输出。⁹
- **GraphRAG、RAPTOR、Long-context RAG 在官方叙事里都不是“万能答案”，而是检索失败模式的补丁包。**GraphRAG 被定义为 RAG 2.0 pipeline 的一个 document preprocessing 环节，不是终局；官方明确提醒它并不适合所有企业数据，也不一定 cost-effective。RAPTOR 默认关闭，因为会消耗 token quota。Long-context RAG 则被表述为保留检索中心地位、通过 TOC/章节上下文弥补 chunking 语义断裂。¹⁰
- **最容易误导人的恰恰是 Solution 页面与模板线索。**这些材料对“行业价值主张”有用，但不足以证明真实落地。更严重的是，法律 Solution 页面出现了明显的金融内容串页：法律页面的示例问题变成了“Analyze NVIDIA’s AI-related revenue changes...” 和销售合规问题，这会直接影响架构判断，必须降权处理。首页上的 Legal precedent analysis 区块也混入了金融投研文案。¹¹

资料清单与场景地图

官方实践案例资料清单

下表把我认为最重要的一手材料按“可复现性”和“工程可学性”做了证据分级。这里的“高”并不等于“生产级”，而是指它已经给到足够多的流程、组件、数据集或 prompt 细节，可以做 fieldbook/labs 复现。相反，Solution 页面与 release notes 模板线索，只能证明产品意图，不能直接当作真实落地方案。¹²

案例名称	原文标题	URL	发布日期	来源类型	行业或业务场景	证据强度	是否有可复现数据集	是否有明确 workflow / agent 结构
投研报告分析 Agent	RAGFlow in Practice - Building an Agent for Deep-Dive Analysis of Company Research Reports	https://ragflow.io/blog/ragflow-in-practice-building-an-agent-for-deep-dive-analysis-of-company-research-reports	2025-10-30	官方实践案例	金融 / 投研 / 研究报告生成	高	是，官方 HF 数据集	是
电商客服 Workflow	Tutorial - Build an E-Commerce Customer Support Agent Using RAGFlow	https://ragflow.io/blog/tutorial-build-an-e-commerce-customer-support-agent-using-ragflow	2025-09-12	官方教程案例	电商客服 / 路由 / 预约收集	高	是，官方 HF 数据集	是

案例名称	原文标题	URL	发布日期	来源类型	行业或业务场景	证据强度	是否有可复现数据集	是否有明确 workflow / agent 结构
SQL Assistant Workflow	Tutorial - Building a SQL Assistant Workflow	https://ragflow.io/blog/tutorial-building-a-sql-assistant-workflow	2025-08-12	官方教程案例	数据分析 / SQL 助手	高	是，官方 HF 数据集	是
RAG-based Text2SQL	Implementing Text2SQL with RAGFlow	https://ragflow.io/blog/implementing-text2sql-with-ragflow	2024-09-24	官方实践案例	Text2SQL / 自动修复循环	中高	是，官方 HF 数据集	结构说明清楚，但偏内置 Agent
Multi-Agent Deep Research	RAGFlow 0.20.0 - Multi-Agent Deep Research	https://ragflow.io/blog/ragflow-0.20.0-multi-agent-deep-research	2025-08-07	官方实践案例	Deep Research / Multi-Agent	中高	否	是
Agentic Workflow 方法论	Agentic Workflow - What's inside RAGFlow 0.20.0	https://ragflow.io/blog/agentic-workflow-whats-inside-ragflow-v0.20.0	2025-08-05	官方实践案例	Workflow vs Agent / 组件哲学	中高	否	是
GraphRAG	How Our GraphRAG Reveals the Hidden Relationships of Jon Snow and the Mother of Dragons	https://ragflow.io/blog/ragflow-support-graphrag	2024-09-19	官方实践案例	GraphRAG / 多跳问答 / QFS	中	否	过程说明清楚
RAPTOR	Implementing a long-context RAG based on RAPTOR	https://ragflow.io/blog/long-context-rag-raptor	2024-05-24	官方实践案例	RAPTOR / long-context retrieval	中	否	有 pipeline 说明

案例名称	原文标题	URL	发布日期	来源类型	行业或业务场景	证据强度	是否有可复现数据集	是否有明确 workflow / agent 结构
Long-context RAG / Ingestion Pipeline	RAGFlow 0.21.0 - Ingestion Pipeline, Long-Context RAG, and Admin CLI	https://ragflow.io/blog/ragflow-0.21.0-ingestion-pipeline-long-context-rag-and-admin-cli	2025-10-15	官方实践案例	Ingestion Pipeline / TOC extraction / admin CLI	中	否	有架构说明
Data Analytics / Sandbox / Memory	RAGFlow 0.25 — Ingestion pipeline, agent sandbox, and user-level memory	https://ragflow.io/blog/ragflow-0.25-ingestion-pipeline-agent-sandbox-and-user-level-memory	2026-04-23	官方实践案例	CodeExec / Data Analytics / user memory	中	未给公开数据集	有模板与流程说明
金融行业方案页	Convert diverse financial data into informed decisions	https://ragflow.io/solutions/financial-services	未标日期	Solution 页面	金融研究 / 合规 / 审计	低	否	只有高层用例
法律行业方案页	Turn contracts and case files into defensible conclusions	https://ragflow.io/solutions/legal-and-compliance	未标日期	Solution 页面	法律 / 合同 / 合规	低	否	只有高层用例
制造行业方案页	Unify manuals and work orders	https://ragflow.io/solutions/manufacturing	未标日期	Solution 页面	制造 / 设备 / 质量	低	否	只有高层用例
教育行业方案页	Build personalized courseware and learning paths	https://ragflow.io/solutions/education	未标日期	Solution 页面	教育 / 备课 / 作业 / 教务	低	否	只有高层用例

案例名称	原文标题	URL	发布日期	来源类型	行业或业务场景	证据强度	是否有可复现数据集	是否有明确 workflow / agent 结构
Deep Research 等模板线索	v0.20.0 / v0.20.1 / v0.20.2 / v0.20.4 / v0.21.0 / v0.22.0 / v0.25.0 release notes	https://ragflow.io/docs/release_notes	2025-08-04 至 2026-04-21	Release notes 模板线索	Deep Research、Text-to-SQL、Choose KB、Report、Ecommerce、Company Research、Interactive、Data Analytics	低	否	仅模板名与一句介绍

案例地图：RAGFlow 官方到底在打哪些场景

金融 / 投研 / 合规

这是 RAGFlow 官方最像“真正在证明路线”的垂直场景。高证据材料包括投研深挖实践文、对应 HF 数据集、以及 release notes 里的 `Company Research Report Deep Dive Agent` 模板线索。其核心业务链路是：自然语言问题 → stock code extraction → 财务 API → 内部研报 retrieval → 差异观点保留式报告生成。金融 Solution 页面还继续往“销售合规、投后跟踪、审计助手”扩展，但这些用例停留在高层描述，没有可复现流程。 ²⁸

法律 / 合规 / 合同

法律方向目前在官方公开材料里**证据最不稳**。唯一专页是法律 Solution 页面，它提出了很像生产系统的目标：按 clause/issue/party 精确定位、附带 statute/precedent、标记 missing support 与 counterexamples；但它后半部分的 use cases 明显串入了金融内容，说明该页存在 copy/paste 或内容治理问题。首页 `Legal precedent analysis` 版块也混入了金融文案。这意味着：**可以学习“法律型可追溯输出”这一产品目标，但不能把该页面当成完整实现证据。** ¹¹

制造 / 设备 / 质量

制造方向的 Solution 页面给出了比较清楚的业务切面：SOP / 设备手册 / 工单 / 质量报告进入统一知识基础，围绕 alarm code、component、station 做 troubleshooting，并把建议 grounding 到原始手册或工单文本，同时回看相似历史事故。它还提出 `告警码 -> 手册章节 -> step-by-step checks -> citations`、`质量异常 -> corrective actions with evidence`、`SOP/ECN -> impacted steps and risks` 这些非常典型的工业知识场景。但目前它仍然只是行业方案页，没有教程、没有模板细节、没有评测。 ²⁵

教育

教育方向也是高层方案而不是完整案例，但其任务拆分比法律更自治：`Lesson prep assistant`、`Homework review assistant`、`Personalized practice`、`Academic affairs Q&A` 都能映射到清晰的数据结构。官方特别强调 explainability：答案要挂到 textbook chapters、curriculum standards，并给错因与修正路径。这说明教育场景在 RAGFlow 叙事里不是单纯问答，而是“知识对齐 + 错因聚类 + 政策引用”。但同样没有复现实验或真实班级级别评测。 ²⁶

电商客服

电商客服是官方最有工程克制的教程。它没有把“客服 Agent”做成大而全的 planning agent，而是先用 `Categorize` 做意图识别，然后分三支：产品参数比较、用户手册问答、安装预约信息收集。两个知识型分支都是 `Retrieval -> Agent`，预约分支则是仅靠 Agent 做多轮 field collection，必要时再接 `HTTP Request` 写回 Google Sheets/Notion。官方还明确警告，heavy Agent 会拖慢响应，简单售后不适合。 ²⁹

数据分析 / SQL / Text2SQL

这一块是仅次于金融的第二条硬证据线。`SQL Assistant Workflow` 给出了最清楚的 `三知识库并行检索 -> SQL 生成 -> SQL Executor` 实践；而 `Implementing Text2SQL` 则给出更抽象的 RAG-based Text2SQL 机制，包括 DDL / Q->SQL / Database Description 三库设计、Loop 自动修复、TopN 限制，以及“最终更稳的生产方案应把标准化处理收口到 API，再封装成 MCP”的方向判断。它证明的不是“RAGFlow 会写 SQL”，而是“Text2SQL 要作为编排系统，而不是单次 prompt”。 ³⁰

Deep Research / 多 Agent

这是 RAGFlow 官方最集中证明 Agent 路线的材料。官方的论点不是“multi-agent 很酷”，而是：Deep Research 这类任务天然包含 decomposition、multi-source retrieval、reflection、iteration，因此不适合复杂拖拽 workflow；真正合适的是由 `Lead Agent` 调度多个 specialized subagent。公开 prompt 里甚至把 URL 数量、搜索策略、工具预算、report structure 都写成了工程约束，远比常见的“一个超级 Agent”叙事更务实。 ⁵

GraphRAG / RAPTOR / Long-context RAG

这三类更像“检索问题专项补丁”而不是行业案例。GraphRAG 处理多跳推理、QFS 与跨 chunk 聚合，但官方明确说它只是 RAG 2.0 pipeline 的一部分，不适合所有企业数据，且并不总是 cost-effective；RAPTOR 通过层级聚类与摘要补跨 chunk 语义，默认关闭以节省 token；Long-context RAG 则通过 TOC extraction / PageIndex 式思路，把章节上下文与 chunk retrieval 结合起来，补传统 chunking 的语义断裂。它们解决的是不同的 retrieval failure mode，而不是谁“更先进”就应该全局替换。 ¹⁰

高价值案例拆解

研报报告分析 Agent

原文地址： <https://ragflow.io/blog/ragflow-in-practice-building-an-agent-for-deep-dive-analysis-of-company-research-reports>

案例类型：官方实践案例。

业务问题：把“查某家公司/股票值不值得买”的分散任务，压缩成一个可自动完成 stock code 识别、财务抓取、内部研报检索与研究报告生成的链路。官方目标语义是：让分析师在数分钟内形成研究判断，而不是手工翻材料。 ³¹

用户输入：典型问题包括“帮我看 Apple 的研究报告”“NVIDIA 财务表现如何”“上证指数今天怎么样”。第一类和第二类可进入链路，第三类因无有效个股 stock code 会走失败分支并返回不支持。 ³²

数据源：内部 `Internal Stock Research Report` 数据集；外部 Tavily Search 用于 stock code extraction；Yahoo Finance 用于财务指标；AlphaVantage 的 `EARNINGS_CALL_TRANSCRIPT` 通过 MCP 工具接入。官方还提供了 InfiniFlow 的 HF 数据集用于复现。 ³³

RAGFlow 组件：Dataset、Paper 解析策略、Agent、Switch/Condition、Yahoo Finance tool、Code、MCP、Retrieval、研究报告生成 Agent、Message。其中 Code 节点负责财务表字段映射与数值格式化，不把表格生成完全交给模型。³⁴

工作流程路径：先用 Agent 做 stock code extraction；若输出不是有效代码，则 Condition 分流到“不支持查询”；如果成功，则用 Yahoo Finance 拉财务数据并交给 Code 格式化 Markdown 表；并行地，信息提取 Agent 调 AlphaVantage + 内部研报 Retrieval Agent；最后由研究报告生成 Agent 依据这两路内容生成结构化研报报告。³⁵

输出物：公司财务表、研报全文抽取、最终投资研究报告。最终报告被要求列出数据来源、机构观点差异、投资建议以及引用信息。官方展示的完整运行耗时约 5 分钟。³⁶

关键 prompt / 规则：最值得学的是三条。第一，stock code extraction 严格限制成“只输出代码或 Not Found”；第二，内部研报 Retrieval Agent 要“完整保留原文、图表描述、风险提示，不可删改”；第三，最终报告生成 Agent 被要求保留不同机构观点的分歧，不准强行合并单一结论。这三条对应三个常见失败点：路由漂移、检索摘要过度压缩、生成阶段抹平差异。³⁷

引用和证据机制：这是官方案例里 citation 设计最成熟的一篇。报告 prompt 明确要求：引用具体数据或观点时标注来源；附录列出 analysis methods 与引用表；机构间有分歧时要保留 author / institution / date。它不只是“显示引用”，而是把 citation 变成“保留分歧和审计线索”的结构化要求。³⁸

工程亮点：第一，内部研报 + 外部财务 API + 外部公开信息 三源合并；第二，检索层与生成层分工清晰；第三，Paper 解析策略适配研报这种非严格标题层级文档；第四，用 Code 固化表格生成，减少模型在财务格式化上的不确定性。³⁹

坏味道或缺口：没有质量评测；没有权限设计；没有说明内外部研报混合时的 source ranking；没有提到人审；5 分钟级时延不适合交互式分析；也没有给出成本控制或失败重试策略。文章证明了“链路能跑通”，但还没有证明“研报团队可安全上线”。⁴⁰

可复现实验：最小可复现集可以直接用官方 HF 中的 2 份 PDF 研报，加一个只读财务 API key。验收标准不要设成“报告写得像券商”，而应设成四项：stock code 提取正确；能拉到财务表；内部研报关键风险提示不丢失；最终报告能显式保留至少两处不同来源的分歧并带 citation。⁴¹

电商客服 Workflow

原文地址：<https://ragflow.io/blog/tutorial-build-an-e-commerce-customer-support-agent-using-ragflow>

案例类型：官方教程案例。

业务问题：解决电商售前/售后里三类最常见但边界清晰的任务：多产品参数对比、用户手册问答、安装预约信息采集。⁴²

用户输入：例如“比较两个型号的功能差异”“某个功能怎么设置”“帮我预约安装”。这是一个很典型的多意图路由应用。⁴³

数据源：两个官方知识库：Product Information 与 User Guide。样例数据可从官方 HF 下载。文档解析选用 Manual chunking，按最小标题保持手册图文完整。⁴⁴

RAGFlow 组件：Begin、Categorize、两个 Retrieval -> Agent 分支、一个纯 Agent 的安装预约分支、可选 HTTP Request、共享 Message。这说明官方把客服案例设计成 workflow-led, agent-assisted，而不是全局 planning agent。⁴⁵

工作流程路径：入口先由 `Categorize` 做意图分类；若是参数比较，则从 `Product Information` 检索后交给 `Feature Comparison Agent`；若是使用指导，则从 `User Guide` 检索后交给 `Usage Guide Agent`；若是安装预约，则直接进入 `Installation Booking Agent` 进行多轮字段收集，之后可接 HTTP 写回外部系统。 44

输出物：比较表或比较要点、分步使用/故障排查说明、安装预约确认信息。预约分支明确要收集三项字段：联系电话、安装时间、安装地址。 45

关键 prompt / 规则：三支分支都很克制。参数比较 Agent 先确认具体型号、保持中立，不做不必要 upsell；手册 Agent 强调 step-by-step 和 common solution first；预约 Agent 强调只追问缺失字段，不重复已收集信息。这里最值得学的不是 prompt 文案本身，而是“每个 Agent 只服务一个狭窄任务目标”。 46

引用和证据机制：这个案例的重点不在 citation，而在正确路由与交互收集。官方没有要求 grounded citations 输出，也没有展示 source-aware response 模板，因此它更像高响应业务流程自动化，而不是强审计场景。 47

工程亮点：`Categorize` 做顶层意图路由，分库分流，避免一个大 Agent 在三个任务之间来回猜；手册类内容采用按结构的 Manual chunking，避免图文拆散；预约写回外部系统通过 `HTTP Request`，而不是让模型“口头承诺已经登记”。 48

坏味道或缺口：没有 SLA、准确率、预约落库成功率等指标；没讨论多轮会话 state 的鲁棒性；也没有处理身份校验、PII 脱敏、预约写入失败重试。官方甚至明确说，HTTP 写回实现细节本文不覆盖。 49

可复现实验：直接用官方 6 份手册 PDF 即可。最小验收标准可设为：三类意图路由准确率、参数比较是否引用到正确型号字段、手册问答是否命中正确章节、预约分支是否在已有部分字段时只追问缺失字段。 50

SQL Assistant Workflow

原文地址：<https://ragflow.io/blog/tutorial-building-a-sql-assistant-workflow>

案例类型：官方教程案例。

业务问题：让非技术用户通过自然语言查询业务数据库，同时把 SQL 生成前的 schema、样例、字段语义知识组织成可检索上下文。 51

用户输入：自然语言业务问题，如查询产品、客户、订单、时序统计等。官方明确将目标用户设定为 marketer、product manager，甚至可作为 SQL 教学工具。 51

数据源：三个知识库——`Schema`、`Question to SQL`、`Database Description`；再加一套真实数据库连接。官方给了 HF 样例数据。这里的数据结构设计本身就是主要工程价值。 52

RAGFlow 组件：`Begin`、三个并行 `Retrieval`、`Agent`、`Execute SQL`、`Message`。无复杂 Agent planning，属于清晰可控的 workflow。 53

工作流程路径：三个知识库并行检索当前 query 的相关信息；`SQL Generator` 汇总三路 `formalized_content` 生成为单条 MySQL；`SQL Executor` 执行并把结果返回给用户。 54

输出物：SQL 语句的执行结果，而不是 SQL 解释文本。Agent prompt 明确要求只输出最终 SQL。 55

关键 prompt / 规则：官方的关键约束是：系统 prompt 规定“单条、语法正确、只返回 SQL”；用户 prompt 把 Schema、Q->SQL 样例、Description 三路内容显式拼入。另一个很值得注意的细节是：Schema 文件建议

避免下划线等特殊字符，因为会诱发 LLM 生成错误 SQL。这个建议不一定普适，但它透露出官方在处理模型脆弱性时是用数据侧规训而不是只靠 prompt。 56

引用和证据机制：此案例几乎不强调 citation；它更像一个 SQL generation/execution pipeline。证据机制体现在“数据库执行是否成功”与“返回结果是否对”上，而不是 grounded citations。 57

工程亮点：三知识库并行检索；Schema 与 Description 采用不同 chunking 规则；Question to SQL 用 Q&A chunking；三库“分维护、分查询、后融合”，避免一个大文本把 schema、语义说明、few-shot 样例混在一起。这个模式高度可迁移。 58

坏味道或缺口：教程没有讨论数据库最小权限、只读连接、SQL allowlist、DML/DDI 禁止、超时与成本；SQL Executor 文档也只是说可以执行 Agent 生成的 SQL，并未在该教程中强调写操作保护。对于真实生产环境，这一块是明显缺口。 57

可复现实验：最小数据集就是官方 Schema.txt、Question to SQL.csv、Database Description EN.txt。验收不要只看“能跑”；应看四项：SQL 可执行率、语义正确率、跨表 join 正确率、错误 SQL 是否会随知识库补充显著下降。 52

RAG-based Text2SQL

原文地址：<https://ragflow.io/blog/implementing-text2sql-with-ragflow>

案例类型：官方实践案例。

业务问题：不用专门 fine-tune Text2SQL 模型，而是把 Text2SQL 做成与 RAG/Agent 可组合的 RAG-based orchestration。 59

用户输入：自然语言数据库查询问题。

数据源：同样依赖三类知识库：DDL、Q->SQL、Database Description；再加一个真实 SQL 执行组件。官方直接把 Text2SQL 视为“多轮编排”问题，而不是“一次生成”问题。 60

RAGFlow 组件：文章时期它把 Text2SQL 封装成内置 Agent；外围则是知识库检索 + Execute SQL。这里更像产品内置能力说明，而不是完整画布教程。 61

workflows 路径：query 先去 Q->SQL 之类的知识源检索相似示例，拼成 prompt 后生成 SQL；执行 SQL；如果执行失败或返回不对，系统依据数据库错误信息自动修复并重试，直到达到 Loop 上限。 62

输出物：数据库结果，或在最大循环次数耗尽后返回失败提示。 63

关键 prompt / 规则：这篇没有公开完整 prompt，但有两个关键设计。第一，知识库是 Text2SQL prompt 的组成部分，而不是 Agent “自由发挥”的补充；第二，Loop 与 TopN 是显式配置，而不是隐式黑箱。 61

引用和证据机制：依赖数据库执行反馈，不是 citation 型 traceability。

工程亮点：把数据库错误信息引入自动修复循环，这是比“只给 schema”更接近生产的设计；同时官方明确说未来更推荐把标准化结构化处理封装成 API，再做成 MCP。这个判断非常重要：它说明官方知道 NL2SQL 不会彻底可靠。 64

坏味道或缺口：没有 benchmark；没有展示修复循环的真实成功率；没有区分查询失败、空结果、歧义问题、权限错误；更没有权限与写安全策略。 65

可复现实验：最小实验应对比“三知识库+反射循环”与“只给 schema”的差异，考察执行成功率、修复轮次、平均 token 成本，而不是只看单题对错。 66

Multi-Agent Deep Research

原文地址： <https://ragflow.io/blog/ragflow-0.20.0-multi-agent-deep-research>

案例类型：官方实践案例。

业务问题：把复杂、开放、多来源的研究任务做成 production-ready 的 Agentic RAG 模板，而不是复杂到不可维护的拖拽 flow。 67

用户输入：开放性研究问题。官方没有限定行业，反而把它当作很多垂直 Agent 的“底座模板”。 68

数据源：内部 RAG、外部 Web Search、MCP 连接、抽取后的 full text。官方要求搜索与抽取都必须通过 tools/MCP 执行，而不是模型脑补。 69

RAGFlow 组件/角色：Lead Agent、Web Search Specialist、Deep Content Reader、Research Synthesizer。这不是“一个大 Agent + 一堆工具”，而是把搜索、阅读、综合三个阶段拆成不同 agent contract。 70

工作流程路径：Lead Agent 先判断 query type，再做 research plan；搜索子 Agent 只交付精选 URL；阅读子 Agent 做 full text extraction 与交叉验证；最后综合子 Agent 基于 ANALYSIS_INSTRUCTIONS 和 EXTRACTED_CONTENT 产出报告。官方把这一链路写成了明确的 stage。 71

输出物：约 2000 词的咨询式研究报告，包含 Executive Summary、Analysis、Recommendations 等部分。官方甚至给到了各 section 的字数预算。 72

关键 prompt / 规则：最值得学的包括：Lead Agent 必须区分 depth-first 与 breadth-first query；搜索子 Agent 必须用工具搜、必须只交 5 个 premium URLs；搜索要平衡 academic / official / industry / news；抽取子 Agent 设有 5-8 次 tool call budget 与 80% extraction success 质量门槛；综合子 Agent 被禁止输出中间处理过程。它背后是一套明确的 agent contract design。 71

引用和证据机制：官方强调 deterministic、grounded outputs，所有模型温度设成 0.1；同时要求 multi-source validation。这里的“grounded citations”更偏向 agent process control，而不是传统 QA 的片段引用 UI。 73

工程亮点：角色分工清楚；避免 attention fragmentation；显式预算与 stopping rule；长上下文模型只放在真正需要 synthesis 的末端；并且允许 BFS/DFS 式规划。这是当前公开材料里最工程化的 multi-agent 设置之一。 74

坏味道或缺口：没有真实 benchmark；80% extraction success 是 prompt 目标，不是报告过的结果；更关键的是，官方自己承认 v0.20.0 当时还不支持人工介入 Deep Research 执行，而这恰恰是 production-readiness 的关键。 75

可复现实验：最小实验不必先接专有内网数据。可以先用 20 个公开研究题，对比：单 Agent、drag-and-drop workflow、official multi-agent template 三者延迟、source diversity、事实冲突保留、final report completeness 上的差异。 76

GraphRAG

原文地址： <https://ragflow.io/blog/ragflow-support-graphrag>

案例类型： 官方实践案例。

业务问题： 解决 naive RAG 在 multi-hop、QFS、跨文档上下文聚合上的失败，特别是“问的是关系网/总结性问题，但 chunk retrieval 只会找局部相似文本”的情况。 ¹⁹

用户输入： 多跳、嵌套逻辑、关系分析类问题；官方用《权游》人物关系做演示。

数据源： 经 Knowledge Graph chunking method 构建的知识图谱，包括命名实体、节点描述和 communities。 ¹⁹

RAGFlow 组件 / 数据结构： 把 knowledge graph construction 定位为 document preprocessing 中的可选步骤；用户需指定要抽取的实体类型，如 organization、person、location。它不是一个单独行业 Agent，而是 retrieval augmentation pipeline。 ¹⁹

工作流程路径： 文档解析时选择 Knowledge Graph 作为 chunking method，定义实体类型，调用 LLM 抽取实体与关系，构图并可视化；随后在 retrieval/chat 阶段使用知识图的 entity / relationship / community 结构辅助回答。 ⁷⁷

输出物： 更深、更全的 multi-hop 问答，以及可视化知识图 / 思维导图。

关键 prompt / 规则： 文章没有公开具体 prompt，但公开了两个关键实现修改：加入 entity deduplication；减少对文档的多次 LLM 提交，以降低 token 消耗。 ¹⁹

引用和证据机制： 与其说是 citation，不如说是结构可解释性。官方明确说 visualization 对 dialogue debugging 很关键，因为图抽取失败会直接导致错误对话。 ¹⁹

工程亮点： 把 GraphRAG 放进 RAG 2.0 pipeline 而非独立神话；明确 entity resolution 与 token cost 是真正的工程难点。

坏味道或缺口： 2024 这篇文章里，GraphRAG 仍然是文档级，不能跨多个文档连图；而且官方明确说它不是最终方案，也不适合对所有企业数据都构图。值得注意的是，到了 2025-02 的 v0.16.0 release notes，官方又宣布 GraphRAG 已改为以整个 dataset 为单位动态构建，并在新文件解析时自动更新。这意味着你在阅读 GraphRAG 材料时必须看版本日期。 ⁷⁸

可复现实验： 最好的最小实验不是开放问答，而是专门做多跳题：例如人物关系链、合同单方义务与例外条款跨节定位、制造事故“故障码—组件—历史案例”的两跳或三跳问答，对比 GENERAL chunking 与 GraphRAG 的支持率。 ¹⁹

RAPTOR 与 Long-context RAG

原文地址： <https://ragflow.io/blog/long-context-rag-raptor> 与 <https://ragflow.io/blog/ragflow-0.21.0-ingestion-pipeline-long-context-rag-and-admin-cli>

案例类型： 官方实践案例 + 产品化演进。

业务问题： 解决跨 chunk 总结、长文档语义断裂，以及“检索到的局部片段不足以回答问题”的 failure mode。 ⁷⁹

用户输入： 更适合跨章节总结、长上下文定位、多步推理问答，而不是单点 factoid retrieval。 ⁷⁹

数据源 / 数据结构： RAPTOR 用树状层级聚类与摘要；RAGFlow 选择 flatten tree retrieval，更适合 multiple recall。Long-context RAG 则通过 TOC extraction，把章节信息挂在 chunk 上，以更接近人类“先看目录再看页”的检索方式。 ⁷⁹

RAGFlow 组件：RAPTOR 是解析后可选开关；Long-context RAG 在 v0.21.0 被产品化为 TOC extraction / PageIndex 方向，并进一步与 Ingestion Pipeline 绑定。Retrieval 组件文档也说明，启用 knowledge graph 会显著增加时间。 80

workflows 路径：RAPTOR 先 chunk，再递归聚类并生成摘要，再将原 chunks + summaries 一并入库；Long-context RAG 在索引期提取章节上下文，在检索期用 TOC 结构补 chunk fragmentation。 79

输出物：仍然是常规 RAG 回答，但 recall 更稳、跨 chunk 支持更强。

关键 prompt / 规则：RAPTOR 这一路没有公开 prompt，但有三个关键工程判断：采用 flatten tree retrieval；默认关闭；需要时再开。Long-context RAG 的关键不是“放弃 retrieval”，而是用 LLM enrich semantics 但保留 indexing/search 中心地位。 79

引用和证据机制：不是 citation 设计，而是 retrieval quality 设计。

工程亮点：官方把 RAPTOR、GraphRAG、Long-context RAG 放在同一族谱里，说明它们都是“语义增强检索”而不是替代检索。v0.21.0 又进一步把这类增强放进可编排的 Ingestion Pipeline。 21

坏味道或缺口：RAPTOR 2024 文章没有 benchmark；默认关闭就意味着成本高；Long-context RAG 在 0.21 仍是 beta；release notes 后来又把 GraphRAG/RAPTOR 的写入方式改成手动 batch build，说明早期自动增量构建在工程上并不理想。 81

可复现实验：适合用“跨 chunk 总结题”“跨章节概览题”“长手册目录定位题”做 A/B：General vs RAPTOR off/on vs TOC extraction。验收要看 answer support rate、跨 chunk 覆盖率与 token/cost 增幅。 79

Ingestion Pipeline 与 Data Analytics 模板

原文地址：<https://ragflow.io/blog/ragflow-0.25-ingestion-pipeline-agent-sandbox-and-user-level-memory>

案例类型：官方实践案例。

业务问题：把“官方内置解析/切块策略不够灵活”和“LLM 不擅长精确数值分析”这两个问题，分别交给可编排 Ingestion Pipeline 与 sandboxed code execution 解决。 82

用户输入：一类是复杂文档处理需求，如金融报表保表格、法律合同做层级切片、简历保留细节；另一类是数字分析需求，如按 12 个月销售数据做增长计算、线性回归、生成图表。 82

数据源：多格式文档、简历、销售数据、用户对话记忆；以及外部系统注入的 sys.user_id。 83

RAGFlow 组件：Ingestion Pipeline 相关组件、增强后的 Title Chunker、CodeExec、Message 中的 User ID 记忆挂接、Retrieval 的 memory 访问。Code 执行需要 Sandbox，官方文档说明其依赖 gVisor 与独立 sandbox backend。 84

workflows 路径：文档侧通过 7 个官方 pipeline templates 复刻内置解析逻辑并允许修改；分析侧通过 Agent 调用 CodeExec 在 sandbox 中执行 Python/JS，生成图表；记忆侧则以 sys.user_id 将 memory 隔离到用户级。 85

输出物：更可控的解析结果、图表 PNG、个性化的 memory-aware 回答。

关键 prompt / 规则：官方最值得记住的不是 prompt，而是判断：LLM 擅长定性，不擅长定量；built-in parsing 可能因为“总结而不是保留原文”丢掉 GPA、细节经历等关键点。于是解析问题交 pipeline，数值问题交 code。 22

引用和证据机制：主要体现在可追溯的原文保留与 per-user memory 隔离，而非 citation UI。

工程亮点：这是 RAGFlow 从“能做 Agent”转向“治理、运维、数据基础”的明显信号。对你做产品架构研究尤其有价值，因为它说明官方已经把 Ingestion Pipeline 与 Code execution 当成核心能力，而不是附属 feature。 ⁸⁶

坏味道或缺口：有 template，但没公开每个模板的完整配置；Data Analytics 只展示效果图，没有精度指标；memory isolation 给了方向，但没有进一步展示永续记忆污染、遗忘策略或跨租户风险控制。 ⁸⁷

可复现实验：最小实验可以先做“简历解析保细节”和“月度销售图表生成”两个小 lab。前者验证 built-in vs pipeline 是否会丢细节，后者验证 code-exec 是否显著优于纯 LLM 计算与图表生成。 ⁸⁸

通用模式与特殊观点

案例背后的通用模式

意图识别与路由

RAGFlow 官方反复用两种路由：Categorize 与 Switch。Categorize 是 LLM-based intent routing，适合“分类边界写得出来但难穷举规则”的场景；官方文档甚至强调，Examples 比 Description 更能帮助分类。Switch 则是 rule-based branching，适合“输出等于什么、是否为空、包含某字符”等确定条件。电商客服用的是前者，投研案例的 stock code 成功/失败分流用的是后者。对工程实践而言，这基本就是“语义路由用 Categorize，契约路由用 Switch”。 ⁸⁹

多知识库并行检索

官方最稳定的工程模式之一，就是按功能或语义角色拆库，而不是按“文档都丢进一个 KB”。SQL 场景拆成 Schema、Q->SQL、Description；电商拆成 Product Information、User Guide；投研则天然分成内部研报、金融 API 返回值、公开搜索结果。Retrieval 文档也支持多 dataset 检索，但要求 embedding model 一致。这个设计的价值在于：每个库可以用不同 chunking 规则、不同更新频率、不同质量标准。 ⁹⁰

RAG + Tool / API

官方实践并不是“只靠知识库”。投研案例里，stock code extraction 用 TavilySearch，财务数据用 Yahoo Finance，公开内容通过 AlphaVantage/MCP 拉取；Deep Research 里，Web Search Specialist 与 Deep Content Reader 都被强制要求通过 tools/MCP 工作；文档层面，RAGFlow 还能作为 MCP server 暴露 retrieve 工具给外部 Agent。此外，HTTP Request component 让 workflow 直接对外系统发请求。换句话说，RAGFlow 的公开路线是：内部数据负责 grounding，外部工具负责 freshness 与 actionability。 ⁹¹

RAG + Code

当事情进入“确定性计算、字段映射、图表输出”时，官方更倾向于 Code / CodeExec。投研案例的财务报表由 Code 节点生成；v0.25 的 Data Analytics 模板干脆把 LLM 定位为分析规划者，而数值计算与可下载图表交给 sandbox 中的 Python。官方文档说明 Code 依赖 gVisor 与 sandbox backend，这也意味着它至少在产品层面认真考虑了隔离。真正值得学的不是“能跑 Python”，而是什么时候不要让模型硬做。 ⁹²

RAG + Structured Output

RAGFlow 官方案例里，structured output 出现得非常频繁。stock code extraction 是单值输出；SQL 助手是单条 SQL；Deep Research 的搜索子 Agent 只输出 5 个 URL；研究综合子 Agent 则必须按 Executive Summary / Analysis / Recommendations 组织结果；安装预约 Agent 则收集 3 个字段。这里值得学的模式是：让每个节点的输出尽量变成 downstream 可消费的 contract，而不是写给人看的大段自然语言。 ⁹³

RAG + Citation / Traceability

金融场景里，citation 的意义是“保留相互冲突的机构观点和出处”；制造场景里，它是“把建议钉回手册章节和工单文本”；教育里，它是“把答案挂回教材章节、课标与错因”；法律方案页则把它表述成“clause-

level / precedent-level 的 defensible conclusion”。但要分清：真正落实到 prompt 级别并公开约束的，主要还是投研案例；Solution 页面更像产品价值主张。 94

Workflow vs Agent

官方最好的判断并不是“Agent 最先进”，而是“Workflows continue to be the primary way Agents are used”，企业里必须混用。Agentic Workflow 一文指出，纯 Agentic Workflow 会引入 unpredictability，尤其对企业不友好；纯 manual Workflow 又容易越拖越乱。于是对稳定、高响应、低歧义场景，用 Workflow；对开放、复杂、研究式任务，用 Agent / Multi-Agent。电商客服文章里对 heavy Agent 响应时间的警告，正是这条判断最落地的实例。 95

Demo 到生产的缺口

官方材料里真正反复出现、但多数教程没有解决的生产缺口包括：权限与 ACL、人工复核、时延、成本、失败恢复、数据新鲜度、删除同步、数据库写安全、API key 管理。金融 Solution 页提到 document-level ACL，但没有实现细节；Deep Research 博文承认没有 human intervention；SQL 教程配置了数据库用户名/密码与执行器，却没讨论 read-only role 或写操作阻断；GraphRAG/RAPTOR 与 rerank/knowledge graph 都被官方承认会显著增加 token 或时间成本。也就是说，官方已经知道生产问题在哪，但公开案例大多还停在“把路径走通”的层面。 96

特殊观点原文清单

下表只列那些可能改变架构判断的点，而不是普通功能介绍。

观点摘要	为什么特殊	原文标题与发布日期	原文地址	建议精读位置	阅读重点	依据
电商客服不适合 heavy Agent，因 planning/ reflection 会拖慢响应	这是少见的官方反 Agent 过度使用的明确表态	Tutorial - Build an E-Commerce Customer Support Agent Using RAGFlow • 2025-09-12	https://ragflow.io/blog/tutorial-build-an-e-commerce-customer-support-agent-using-ragflow	Summary 段	评估你的客服场景是否真的需要 Agent	4
投研链路不是单纯检索，而是 stock code -> finance API -> internal report -> report generation	它展示了 RAGFlow 官方如何把 Context Engine 做成跨源编排	RAGFlow in Practice - Building an Agent for Deep-Dive Analysis of Company Research Reports • 2025-10-30	https://ragflow.io/blog/ragflow-in-practice-building-an-agent-for-deep-dive-analysis-of-company-research-reports	2.2-2.5	关注外部 API 与内部检索如何分工	97
最终投研报告必须保留不同机构分歧，不能强行合并结论	这是比“加 citation”更强的生成约束	同上	同上	Research Report Generation Agent prompt	关注 conflict preservation，而不是 summary fluency	36

观点摘要	为什么特殊	原文标题与发布日期	原文地址	建议精读位置	阅读重点	依据
SQL / Text2SQL 要靠三类知识库，而不是只给 schema	这是官方最稳定、最可迁移的数据结构模式	Tutorial - Building a SQL Assistant Workflow • 2025-08-12; Implementing Text2SQL with RAGFlow • 2024-09-24	https://ragflow.io/blog/tutorial-building-a-sql-assistant-workflow ; https://ragflow.io/blog/implementing-text2sql-with-ragflow	KB 准备与 User Prompt	判断你的 SQL 场景需不需要样例库和语义描述库	98
Text2SQL 失败后要依据数据库错误信息自动修复并重试	这比单轮 SQL 生成更接近生产	Implementing Text2SQL with RAGFlow • 2024-09-24	https://ragflow.io/blog/implementing-text2sql-with-ragflow	Configure Loop / Troubleshooting	关注 loop 的成功率与上限策略	63
Deep Research 更适合 Agentic 方法，不适合复杂拖拽 workflow	官方明确给出“为什么 workflow 只够 demo”的理由	RAGFlow 0.20.0 - Multi-Agent Deep Research • 2025-08-07	https://ragflow.io/blog/ragflow-0.20.0-multi-agent-deep-research	开头对比 workflow vs agentic	关注复杂度、可维护性与控制逻辑	67
Deep Research 搜索子 Agent 被强制要求只返回 5 个 premium URLs	这是典型的 attention budget 管理	同上	同上	Web Search Specialist prompt	看官方如何把“别搜太多”写成 contract	99
Deep Research 支持 BFS/DFS 式计划	说明它不是线性 workflow 的轻改版	同上	同上	Lead Agent prompt	评估你的任务是否真需要 planning search tree	100
Deep Research 当前版本不支持 human intervention	官方自己承认 production-ready 还差人审/人工纠偏	同上	同上	Upcoming versions	这直接影响是否能上高风险场景	101

观点摘要	为什么特殊	原文标题与发布日期	原文地址	建议精读位置	阅读重点	依据
GraphRAG 只是 RAG 2.0 pipeline 的一个 preprocessing 部分，不是终局	避免把 GraphRAG 神化	How Our GraphRAG Reveals the Hidden Relationships of Jon Snow and the Mother of Dragons • 2024-09-19	https://ragflow.io/blog/ragflow-support-graphrag	开头定义 RAG 2.0	判断 GraphRAG 在你的栈里应该放在哪一级	19
GraphRAG 不适合所有企业数据，也不一定 cost-effective	这是非常重要的成本边界提醒	同上	同上	总结段	不要默认全库建图	19
RAPTOR 默认关闭，因为会消耗 token quota	说明官方把它当高成本增强项，而不是默认 best practice	Implementing a long-context RAG based on RAPTOR • 2024-05-24	https://ragflow.io/blog/long-context-rag-raptor	RAPTOR switch 段	先评估收益，再决定是否开启	20
Long-context RAG 不是放弃 retrieval，而是用 TOC 丰富语义后继续检索	这体现了官方“检索仍是中心”的立场	RAGFlow 0.21.0 - Ingestion Pipeline, Long-Context RAG, and Admin CLI • 2025-10-15	https://ragflow.io/blog/ragflow-0.21.0-ingestion-pipeline-long-context-rag-and-admin-cli	Long-context RAG 段	注意它与 grep/Agentic RAG 的区别	102
Ingestion Pipeline 被定位成非结构化数据领域的 ETL/ELT	这是 RAGFlow 从产品到基础设施的方向变化	同上	同上	Ingestion Pipeline 段中 ETL/ELT 比喻	关注 Parser/Transformer/Indexer 的职责分离	103
LLM 不擅长定量分析，应该用 sandbox code execution	官方公开把“不要让模型硬算”说透了	RAGFlow 0.25 — Ingestion pipeline, agent sandbox, and user-level memory • 2026-04-23	https://ragflow.io/blog/ragflow-0.25-ingestion-pipeline-agent-sandbox-and-user-level-memory	Agent sandbox execution and charting	关注 CodeExec 进入业务后的位置	104

观点摘要	为什么特殊	原文标题与发布日期	原文地址	建议精读位置	阅读重点	依据
built-in parsing 可能因“总结而不是保留原文”丢掉 GPA 等细节	这是对解析设计的很具体警告	同上	同上	Ingestion pipeline enhancement 中的 resume parsing 对比	关注“保摘要”与“保原文”的 trade-off	105
法律 Solution 页面出现明显金融串页	这是证据治理问题，不只是文案错误	Turn contracts and case files into defensible conclusions • 未标日期	https://ragflow.io/solutions/legal-and-compliance	Use cases 段	读任何 Solution 页面都要先判断是否自治	24

可验证实验与学习路线

可验证实验设计

实验名称	来源案例	要验证的官方设计判断	最小数据集	实现方式	对照组	指标	验证
电商客服路由实验	电商客服 Workflow 42	Categorize + 轻 Agent 比 heavy Agent 更适合高响应客服	官方 6 份手册 + 120 条人工标注 query	一版按官方 workflow; 一版做成单一 planning Agent	Heavy Agent 全局路由	路由准确率、平均延迟、token 成本、预约字段收集完整率	wo 延 低 劣 Ag
Text2SQL 三知识库实验	SQL Assistant + Text2SQL 106	三知识库优于只给 schema	官方 text2sql 样例库 + 一只读 MySQL	按官方三库并行检索与执行	只给 schema、无 Q->SQL、无 DB Description	SQL 可执行率、语义正确率、平均修复轮次、token 成本	三 jo 义 优
投研报告融合实验	投研报告分析 Agent 107	内部研报 + 外部财务 API + 差异观点保留，优于单一检索摘要	官方 2 份研报 + Yahoo/AlphaVantage 接口	复刻官方链路	只检索内部研报 / 只用外部 API	citation 覆盖率、分歧保留率、事实遗漏率、生成耗时	至 来 留 提
GraphRAG 多跳问答实验	GraphRAG 19	多跳问题上 GraphRAG 优于 General chunking	20 篇人物/合同/设备案例文档 + 50 道多跳题	一版 Knowledge Graph chunking, 一版 General	General chunking	multi-hop answer support rate、平均检索步数、延迟	Gr 多 明

实验名称	来源案例	要验证的官方设计判断	最小数据集	实现方式	对照组	指标	验证
RAPTOR 跨 chunk 总结实验	RAPTOR + Long-context RAG ⁷⁹	RAPTOR/TOC extraction 能提高跨 chunk 总结题	10 篇长文档，每篇 > 30 页	General / RAPTOR off / RAPTOR on / TOC extraction 四组	General chunking	summary factuality、跨章节覆盖率、token 成本、延迟	开盖本
制造手册故障检索实验	制造 Solution 页 ²⁵	alarm code -> 手册章节 -> grounded answer 是有效模式	3 本设备手册 + 100 条告警码问答	建立设备知识库 + section-aware retrieval	纯全文 semantic search	正确章节命中率、citation 支持率、step-by-step 完整率	正率
法律合同反例检索实验	法律 Solution 页 ²⁴	“高亮 missing support / surfacing counterexamples” 是否能降低过度自信	20 份合同 + 50 条风险提问	clause-level 检索 + counterexample prompt	只做正向条款检索	反例召回率、unsupported claim rate、审查时间	加 un cl 降
教育错因聚类实验	教育 Solution 页 ²⁶	错因聚类和个性化练习未必需要 Agent，workflow 也许够用	500 条作业记录 + 标准答案 + 课标	一版 workflow 聚类/推荐，一版 Agent 规划	纯 retrieval / 普通规则	错因聚类 purity、个性化推荐命中率、延迟、教师满意度	w 到 且

对你的学习路线建议

先精读哪些原文

如果你时间有限，我建议先读五篇：

1. RAGFlow in Practice - Building an Agent for Deep-Dive Analysis of Company Research Reports，因为它最能代表 RAGFlow 官方如何把 RAG、外部 API、MCP、Code、citation 组合成有业务价值的完整链路。¹⁰⁸
2. Tutorial - Building a SQL Assistant Workflow，因为它把“多知识库并行检索”的结构表达得最清楚。¹⁰⁹
3. Implementing Text2SQL with RAGFlow，因为它把自动修复循环和生产边界说透了。⁶³
4. RAGFlow 0.20.0 - Multi-Agent Deep Research，因为这是 Agent 路线的核心证据。¹¹⁰
5. Agentic Workflow - What's inside RAGFlow 0.20.0，因为它决定你何时该上 Workflow、何时该上 Agent。¹⁸

哪些案例只扫一遍即可

四个 Solution 页面目前都只值得扫一遍判断方向，不能拿它们当实现蓝图。尤其法律页，必须带着“证据降权”去看。¹¹¹

哪些案例适合马上进入 fieldbook 笔记

适合立刻做成可复用 note 的有三类：

- 一类是“结构模式”，如 SQL 三知识库、投研中的内外源合并、Deep Research 的角色拆分。¹¹²
- 一类是“反常识判断”，如客服不该 heavy Agent、RAPTOR 默认关闭、GraphRAG 不适合全量数据。¹¹³
- 一类是“产品方向变化”，如 0.21 起把 Ingestion Pipeline 做成非结构化 ETL/ELT，0.25 起把 CodeExec 与 user-level memory 拉进核心叙事。¹¹⁴

哪些适合进入 labs

最值得立刻做 lab 的是：电商 workflow vs heavy Agent、Text2SQL 三库 vs 单 schema、投研三源融合、GraphRAG 多跳问答、RAPTOR/TOC extraction 跨 chunk 总结。它们都能做清晰对照，而且成功/失败都能产出可靠结论。 115

哪些暂时不要做，避免过度工程

我不建议你一开始就做全量 legal precedent analysis、多租户 ACL、全图 GraphRAG 或长链 Deep Research 上生产。原因不是这些不重要，而是官方公开材料对这些部分仍缺乏实现细节、评测和治理说明。先做可验证的小实验，比直接仿一个“大平台”更划算。 116

如果只能做一个最小实验

我会选 **Text2SQL：三知识库 RAG Text2SQL vs 只给 schema**。原因有三点：第一，数据和环境最容易搭；第二，指标最容易量化；第三，它能同时验证你对“分库结构”“检索增强”“执行反馈修复循环”的理解，外推价值比单一 QA 场景更大。 117

结论与参考资料

结论

RAGFlow 官方实践案例最值得学习的五个模式

1. **按任务角色拆数据，而不是把所有文档混成一个知识库**。SQL 的 Schema / Q->SQL / Description，电商的 Product / User Guide，投研的内外源分层，都是同一个模式。 118
2. **把 Agent 输出收窄成 contract**。stock code、5 个 URL、单条 SQL、预约三字段，都是“先缩窄，再放大”。 119
3. **把 Tool/API 与内部 RAG 并列成一等公民**。鲜度靠外部，grounding 靠内部，执行靠 SQL/HTTP/MCP。 120
4. **把 Code 放到必须 deterministic 的位置**。表格、计算、图表，不和 LLM 争。 121
5. **把 Workflow 与 Agent 明确分工**。稳定场景走 workflow，复杂研究走 multi-agent，不强推全 Agent 化。 122

官方案例最容易误导初学者的五个点

1. 把 Solution 页面误当成已验证落地案例。实际上它们大多没有流程细节、没有评测、没有数据集。 111
2. 看到模板名就以为有完整实现。release notes 里的 template 多数只是方向线索。 123
3. 误以为 GraphRAG / RAPTOR / Long-context RAG 是默认 best practice。官方其实一直在强调成本和适用边界。 124
4. 看到 SQL 能跑通 demo，就误判为可直接生产。官方教程并没有补齐权限、安全、写操作阻断等制度层设计。 57
5. 误以为 Deep Research 已经 production-ready。官方自己承认还缺 human intervention。 101

哪些场景，RAGFlow 的设计确实有好品味

金融投研、SQL/Text2SQL、电商客服这三类，公开材料体现出的设计品味最好。投研胜在多源合并与分歧保留；SQL 胜在三知识库结构；电商客服胜在对 heavy Agent 的克制。Deep Research 的 prompt-level contract 设计也非常值得学，但它更像高级模式，不该在团队还没掌握基础 workflow 前就直接照搬。 125

哪些场景还只是 demo，不能当生产方案

法律、制造、教育目前在官方公开材料里仍主要是高层行业方案；GraphRAG、RAPTOR、Long-context 更多是技术演示；0.25 的 Data Analytics 模板也只证明“代码沙箱这条路合理”，没证明业务级数据分析助手已经稳定。它们能作为学习方向，但还不能直接当 production architecture 蓝本。 ¹²⁶

你下一步最该啃的五篇原文

1. RAGFlow in Practice - Building an Agent for Deep-Dive Analysis of Company Research Reports。 ¹²⁶
2. Tutorial - Building a SQL Assistant Workflow。 ¹²⁷
3. Implementing Text2SQL with RAGFlow。 ¹²⁸
4. RAGFlow 0.20.0 - Multi-Agent Deep Research。 ¹²⁹
5. Agentic Workflow - What's inside RAGFlow 0.20.0。 ¹³⁰

这五篇加起来，基本覆盖了 RAGFlow 官方目前最重要的工程判断：数据结构、workflow/agent 边界、外部工具接入、structured output、反射/多 Agent、以及 traceability。 ¹²⁷

Open questions / limitations

本次结论有几个必须明说的限制。第一，法律、制造、教育方向缺少和投研/SQL 同级别的教程或实践文，因此很多判断只能停留在“产品意图”层面。第二，多数官方案例仍缺 benchmark，很多“效果更好”的判断来自截图或方法论，而不是规范评测。第三，一些页面存在内容污染或版本漂移，最典型的是法律 Solution 页与 GraphRAG 从 2024 到 2025 的能力变化，因此阅读时必须把“发布日期”和“版本号”绑定起来。 ¹²⁸

完整参考资料列表

- RAGFlow 官方博客索引：<https://ragflow.io/blog> ¹²⁹
- 官方实践案例：<https://ragflow.io/blog/ragflow-in-practice-building-an-agent-for-deep-dive-analysis-of-company-research-reports> ¹³⁰
- 官方教程案例：<https://ragflow.io/blog/tutorial-build-an-e-commerce-customer-support-agent-using-ragflow> ¹³¹
- 官方教程案例：<https://ragflow.io/blog/tutorial-building-a-sql-assistant-workflow> ¹³²
- 官方实践案例：<https://ragflow.io/blog/implementing-text2sql-with-ragflow> ¹³³
- 官方实践案例：<https://ragflow.io/blog/ragflow-0.20.0-multi-agent-deep-research> ¹³⁴
- 官方实践案例：<https://ragflow.io/blog/agentic-workflow-whats-inside-ragflow-v0.20.0> ¹⁸
- 官方实践案例：<https://ragflow.io/blog/ragflow-support-graphrag> ¹⁹
- 官方实践案例：<https://ragflow.io/blog/long-context-rag-raptor> ²⁰
- 官方实践案例：<https://ragflow.io/blog/ragflow-0.21.0-ingestion-pipeline-long-context-rag-and-admin-cli> ¹³⁵
- 官方实践案例：<https://ragflow.io/blog/ragflow-0.25-ingestion-pipeline-agent-sandbox-and-user-level-memory> ¹³⁶
- 金融 Solution 页面：<https://ragflow.io/solutions/financial-services> ²³
- 法律 Solution 页面：<https://ragflow.io/solutions/legal-and-compliance> ²⁴
- 制造 Solution 页面：<https://ragflow.io/solutions/manufacturing> ²⁵
- 教育 Solution 页面：<https://ragflow.io/solutions/education> ¹³⁷
- 官方文档总览：<https://ragflow.io/docs/> ¹³⁸
- Components 文档：<https://ragflow.io/docs/category/components> ¹³⁹
- Agent component：https://ragflow.io/docs/agent_component ¹⁴⁰
- Retrieval component：https://ragflow.io/docs/retrieval_component ¹⁴¹
- Categorize component：https://ragflow.io/docs/categorize_component ¹⁴²
- Switch component：https://ragflow.io/docs/v0.25.1/switch_component ¹⁴³
- Code component：https://ragflow.io/docs/code_component ¹⁴⁴

- Execute SQL tool: https://ragflow.io/docs/execute_sql 145
- HTTP request component: https://ragflow.io/docs/http_request_component 146
- MCP 分类与工具: <https://ragflow.io/docs/category/mcp>、https://ragflow.io/docs/mcp_tools、https://ragflow.io/docs/launch_mcp_server 147
- Release notes: https://ragflow.io/docs/release_notes 148
- GitHub 仓库: <https://github.com/infiniflow/ragflow> 149
- GitHub releases: <https://github.com/infiniflow/ragflow/releases> 150
- 官方 HF 数据集: https://huggingface.co/datasets/InfiniFlow/company_financial_research_agent、<https://huggingface.co/datasets/InfiniFlow/Ecommerce-Customer-Service-Workflow>、<https://huggingface.co/datasets/InfiniFlow/text2sql/tree/main> 151

1 2 7 8 9 28 31 32 33 34 35 36 37 38 39 91 92 93 94 97 107 108 119 120 121 125 127 130

<https://ragflow.io/blog/ragflow-in-practice-building-an-agent-for-deep-dive-analysis-of-company-research-reports>

<https://ragflow.io/blog/ragflow-in-practice-building-an-agent-for-deep-dive-analysis-of-company-research-reports>

3 18 95 122 **Agentic Workflow - What's inside RAGFlow 0.20.0 | RAGFlow**

<https://ragflow.io/blog/agentic-workflow-whats-inside-ragflow-v0.20.0>

4 14 29 41 42 43 44 45 46 47 48 49 113 115 131 **Tutorial - Build an E-Commerce Customer Support Agent Using RAGFlow | RAGFlow**

<https://ragflow.io/blog/tutorial-build-an-e-commerce-customer-support-agent-using-ragflow>

5 67 68 69 70 71 72 73 74 75 76 99 100 101 110 134 **RAGFlow 0.20.0 - Multi-Agent Deep Research | RAGFlow**

<https://ragflow.io/blog/ragflow-0.20.0-multi-agent-deep-research>

6 30 51 52 53 54 55 56 57 58 90 98 106 109 112 117 118 **Tutorial - Building a SQL Assistant Workflow | RAGFlow**

<https://ragflow.io/blog/tutorial-building-a-sql-assistant-workflow>

10 19 77 78 124 **How Our GraphRAG Reveals the Hidden Relationships of Jon Snow and the Mother of Dragons | RAGFlow**

<https://ragflow.io/blog/ragflow-support-graphrag>

11 24 116 126 128 **<https://ragflow.io/solutions/legal-and-compliance>**

<https://ragflow.io/solutions/legal-and-compliance>

12 13 129 **Blog**

https://ragflow.io/blog?utm_source=chatgpt.com

15 132 **Tutorial - Building a SQL Assistant Workflow**

https://ragflow.io/blog/tutorial-building-a-sql-assistant-workflow?utm_source=chatgpt.com

16 59 60 61 62 63 64 65 66 **Implementing Text2SQL with RAGFlow | RAGFlow**

<https://ragflow.io/blog/implementing-text2sql-with-ragflow>

17 **RAGFlow 0.20.0 - Multi-Agent Deep Research**

https://ragflow.io/blog/ragflow-0.20.0-multi-agent-deep-research?utm_source=chatgpt.com

20 79 80 81 **Implementing a long-context RAG based on RAPTOR | RAGFlow**

<https://ragflow.io/blog/long-context-rag-raptor>

21 102 103 114 135 <https://ragflow.io/blog/ragflow-0.21.0-ingestion-pipeline-long-context-rag-and-admin-cli>
<https://ragflow.io/blog/ragflow-0.21.0-ingestion-pipeline-long-context-rag-and-admin-cli>

22 82 83 84 85 86 87 88 104 105 136 <https://ragflow.io/blog/ragflow-0.25-ingestion-pipeline-agent-sandbox-and-user-level-memory>
<https://ragflow.io/blog/ragflow-0.25-ingestion-pipeline-agent-sandbox-and-user-level-memory>

23 96 111 <https://ragflow.io/solutions/financial-services>
<https://ragflow.io/solutions/financial-services>

25 <https://ragflow.io/solutions/manufacturing>
<https://ragflow.io/solutions/manufacturing>

26 137 <https://ragflow.io/solutions/education>
<https://ragflow.io/solutions/education>

27 123 148 [Releases | RAGFlow](https://ragflow.io/docs/release_notes)
https://ragflow.io/docs/release_notes

40 151 https://huggingface.co/datasets/InfiniFlow/company_financial_research_agent
https://huggingface.co/datasets/InfiniFlow/company_financial_research_agent

50 <https://huggingface.co/datasets/InfiniFlow/Ecommerce-Customer-Service-Workflow>
<https://huggingface.co/datasets/InfiniFlow/Ecommerce-Customer-Service-Workflow>

89 142 https://ragflow.io/docs/categorize_component
https://ragflow.io/docs/categorize_component

133 [Implementing Text2SQL with RAGFlow](https://ragflow.io/blog/implementing-text2sql-with-ragflow?utm_source=chatgpt.com)
https://ragflow.io/blog/implementing-text2sql-with-ragflow?utm_source=chatgpt.com

138 [Quickstart](https://ragflow.io/docs/?utm_source=chatgpt.com)
https://ragflow.io/docs/?utm_source=chatgpt.com

139 [Components | RAGFlow](https://ragflow.io/docs/category/components)
<https://ragflow.io/docs/category/components>

140 [Agent component | RAGFlow](https://ragflow.io/docs/agent_component)
https://ragflow.io/docs/agent_component

141 https://ragflow.io/docs/retrieval_component
https://ragflow.io/docs/retrieval_component

143 https://ragflow.io/docs/v0.25.1/switch_component
https://ragflow.io/docs/v0.25.1/switch_component

144 https://ragflow.io/docs/code_component
https://ragflow.io/docs/code_component

145 https://ragflow.io/docs/execute_sql
https://ragflow.io/docs/execute_sql

146 https://ragflow.io/docs/http_request_component
https://ragflow.io/docs/http_request_component

147 [MCP](https://ragflow.io/docs/category/mcp?utm_source=chatgpt.com)
https://ragflow.io/docs/category/mcp?utm_source=chatgpt.com

149 GitHub - infiniflow/ragflow: RAGFlow is a leading open-source Retrieval-Augmented Generation (RAG) engine that fuses cutting-edge RAG with Agent capabilities to create a superior context layer for LLMs • GitHub

<https://github.com/infiniflow/ragflow>

150 <https://github.com/infiniflow/ragflow/releases>

<https://github.com/infiniflow/ragflow/releases>